

Outline Font Generating from Images of Ancient Chinese Calligraphy

Junsong Zhang¹, Guohong Mao¹, Hongwei Lin², Jinhui Yu², and Changle Zhou¹

¹ Lab of Mind, Art&Computation, Cognitive Science Department,
School of Information Science and Technology, Xiamen University, Xiamen 361005, China
{zhangjs, maogh, dozero}@xmu.edu.cn

² State Key Lab of CAD&CG, Zhejiang University, Hangzhou 310058, China
{hwlin, jhyu}@cad.zju.edu.cn

Abstract. Chinese calligraphy is an art unique to Asian cultures. This paper presents a novel method for generating outline font from historical documents of Chinese calligraphy. The method consists of detecting feature points from character boundaries, and approximating contour segments. The feature-point-detection is based on statistical method considering the characteristics of a calligrapher. A database of basic strokes and some overlapping stroke components of Chinese characters extracted from the calligrapher are constructed in advance. And the relation between the noise level of stroke contours and the standard deviation of Gaussian kernel is retrieved from the database using linear regression. Thus, given an input character contour, the standard deviation for smoothing the noisy character contour can be calculated. Furthermore, a new method is employed to determine the feature points at the standard deviation. The feature points at a character contour subdivide the contour into segments. Each segment can be fitted by a parametric curve to obtain the outline font. Some experimental results and the comparisons to existing methods are also presented in the paper.

Keywords: Chinese Calligraphy, Historical Document, Outline Font, Feature Point Detection.

1 Introduction

Chinese calligraphy has a long history of more than four thousand years, there are abundant historical documents of calligraphy left and stored in the libraries and museums (Fig.1 shows two examples). The calligraphy documents stored as images are not easy for further application. That is because each Chinese character is stored as an array of pixels, which can be seen as bitmap font, and the scaling operation for the character images will bring significant distortion effect. In addition, recording images of calligraphy document at pixel level also requires a lot of storage space [1].

In contrast to the bitmap font, the outline font, such as TrueType font [2], defined as a set of parametric curves, can be scaled to any size and otherwise transformed more easily than a bitmap font through certain numerical processing. It would be desirable therefore to convert the Chinese characters from bitmap font into outline font.

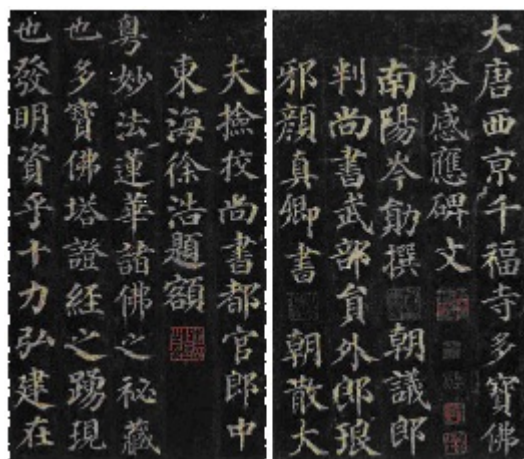


Fig. 1. Calligraphy tablet images from Zhenqing Yan (A.D.709 - 785)

However, due to long storage time of the original paper documents, uneven color of the document surfaces, as well as some noise generated by the scanning, the scanned document images of Chinese calligraphy are noisy. As a result, the detected character boundaries from the document images are usually rough, the feature points are thus hard to be detected accurately. Research on feature-point-detection from planar shapes has been conducted for a couple of decades, and many methods have been proposed [3]. These methods mainly fall into two categories, CSS (curvature scale space) based methods [4][5] and ROS (region of support) based methods [6][7].

The ROS based algorithms for feature-point-detection mainly depend on the decision of a region of support for every point on the curve, and the ROS should reflect the local property of the feature point. Teh and Chin [8] suggested that the detection of feature points relies primarily on the precise determination of the ROS. To support their argument, they applied three different significance measures (k-curvature, 1-curvature and k-cosine) to different test shapes, and showed that once the ROS is determined properly, different significance measures produce similar results. However, the ROS is difficult to accurately determine, especially when the contour is noisy. And most of the previous methods need a number of additional thresholds and parameters for determining the ROS. Hence, the traditional ROS methods will not be considered here because of our need for a robust method with as few parameters as possible.

The scale space concept was introduced by Witkin [9]. It has been widely extended to many research fields. The idea of scale space is that objects in nature present us with different scales, or sizes. From the viewpoint of shape analysis, the CSS method is essentially a multi-scale organization of invariant geometric features of a planar contour. The features consist of curvature zero-crossings or local extrema obtained from the smoothed curvature function at multiple scales. And curvature function smoothing is achieved by convolving the curvature function using a Gaussian kernel with increasing standard deviation. Since multi-scale organization of the curvature function results in a natural representation of shape information at various levels of

details, where noise and insignificant features are filtered out at smaller scales and only the prominent shape features surviving to larger scales, the CSS representation is therefore robust to noise. However, the CSS based feature-point-detection algorithm has been suggested that if too large a scale is selected, some feature points of fine features will be missed, and vice versa [8]. It is also computationally intensive due to the iterative convolutions, and some locations of the detected feature points on the curve are inaccurate because of the smoothing effect by Gaussian kernel.

In this paper, we propose a statistical deduction based algorithm for detecting feature point on the noisy character contours extracted from historical calligraphy images. The core idea is to obtain the relationship between the noise level of character contour and the standard deviation of Gaussian function by use of linear regression. Given the curvature sum of a new character contour, a standard deviation for smoothing the curvature function of character contour can be calculated with the linear regression formula, all local extrema computed from the smoothed curvature function are recorded as feature point candidates. Furthermore the final feature points from the character contour is chosen by employing a novel method. These feature points are then used to divide the whole contour into segments, and each segment is fitted with a parametric curve according to their bending strength, resulting the outline font of the corresponding input character image.

The key advantages of our method are that it is faster than CSS and more accurate than ROS for feature-point-detection. The calligraphic characteristics of the original characters is retained and the noises are removed from the original character outline.

The remainder of this paper is organized as follows. In Section 2, the method for character outline extraction and contour tracing is introduced. In Section 3, we propose a statistical deduction based feature-point-detection algorithm for character contours. In Section 4, a least square based curve approximating method is presented to express the character contours in outline font form. And some experimental results are given in Section 5. Finally, Section 6 draws a brief conclusion.

2 Character Outline Extracting and Tracing

To extract the outline of characters from calligraphy documents images, we choose Canny operator [10] as the outline detector.

Let I denotes an input binary character image, after the character outline is extracted, the value of outline pixels is set 0 and the value of other pixels is set 1 to get a binary image I_c . We save all pixels with 0 value in a list $L(l_1, l_2, \dots, l_k)$, and record the pixels of each closed contour in l_k in clockwise direction according to the result of contour tracing. To trace a closed contour, we scan the pixels of I_c from bottom to top and left to right until a black pixel s is found, s is taken as the start point for contour tracing. Then we find the next black pixel p adjacent to s in the clockwise direction, and take p as the current start point for tracing the next pixel. The tracing process terminates when s is visited again.

Most of Chinese characters consist of multiple strokes, thus we may get several closed contours after the character outline is extracted. In the case of multiple closed contour need to be traced, we use a flag f for every contour point, if a pixel is traced, its flag f is set to be 1, otherwise the flag f is set to be 0. The use of flag in our system ensures that all closed contours will be traced.

3 Feature Points Determination

3.1 Constructing Statistical Samples

Although Chinese characters in the calligraphy documents are numerous, each character is in fact represented by the combination of basic strokes. A basic stroke is a single stroke but varies in shape. Some typical basic strokes are point, horizontal, vertical, left slant and right slant. A Chinese character may contain basic strokes ranging from 1 to about 30, in which some basic strokes may overlap each other to form different components. These basic strokes and overlapping stroke components may result in closed contour corresponding to their outlines. On these closed contours, basic strokes have their own feature points due to their shapes, and overlapping stroke components have more feature points at the junction of basic strokes. To construct the sample database for evaluating the relationship between the noise level of their outlines and the standard deviation of Gaussian kernel, we should take both basic strokes and overlapping stroke components into account.

Since calligraphers have written Chinese characters with different ways, various styles of Chinese calligraphy thus formed. To process various calligraphy images of different calligraphers, we first construct a statistical sample from their basic strokes and components respectively. In this paper, we choose 120 representative strokes and components from the masterpiece by a famous Chinese calligrapher Mr. Zhenqing Yan (A.D.709 - 785) to construct the sample database.

3.2 Linear Regression to Determine Feature Point Candidates

After the sample database is constructed, we extract their outlines and calculate the absolute curvature for each point on the closed contour to obtain a curvature function with reference to the contour length. The curvature function is then convolved with a Gaussian kernel. The convolution will smooth the curvature function, and the standard deviation of Gaussian kernel is selected according to the noise level of stroke contour by users' interaction. Generally, the Gaussian function is defined as

$$g(s, \delta) = \frac{1}{\delta\sqrt{2\pi}} e^{\frac{-s^2}{2\delta^2}} \quad (1)$$

where δ is the standard deviation of Gaussian kernel, acting as the scale. The convolution at the scale δ between the curvature function and the Gaussian function is

$$C(l, \delta) = \int_{u=l-\frac{L}{2}}^{u=l+\frac{L}{2}} c(u) g(l-u, \delta) du \quad (2)$$

here, L is the total length of the curvature curve, $c(u)$ is the absolute curvature at the contour point, and $C(l, \delta)$ is the convolution curvature curve at the scale.

According to our experiment, the selected standard deviation δ for smoothing a given character contour is related to the curvature sum of the character contour. In Fig.2, this is illustrated, Fig.2(a) is an original stroke contour with curvature sum 3.3456. To simulate the character contour with various levels of random noise, we

add different uniform-random-noise to Fig.2 (a) to derive Fig. 2(b)-(e) with variance $V=1, 2, 3, 4$ respectively. Here the bigger variance added to the stroke contour will lead to the stroke contour with more noise. This suggests that for contours of the same character, bigger curvature sum indicates higher noise level of the stroke contour. To eliminate the influence of noise for the detection of feature points, a bigger standard deviation of Gaussian kernel is needed to smooth the noisy stroke contour. Hence the key is to reveal the relation between the curvature sum and the standard deviation of Gaussian kernel. For each contour in the sample contour database, we compute the curvature sum by adding curvatures of all contour points on a single closed contour together, and select the optimal scale δ by user interaction. Taking the curvature sum as x-coordinates, the scale as y-coordinates, a discrete point set (x_i, y_i) , $i=1, 2, \dots, n$, is obtained in the x - y plane. The relation between the curvature sum and the scale can be constructed using the linear regression model [11].

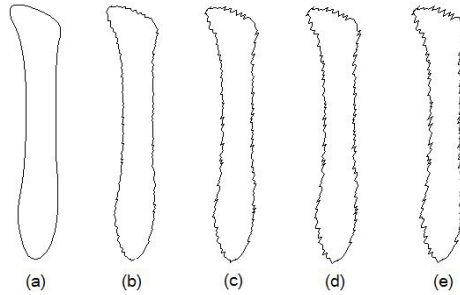


Fig. 2. A stroke contour with different variance of uniform random noise, from left to right: (a) original contour, (b)-(e) results of adding uniform random noise to (a) with variance $V=1$, $V=2$, $V=3$, and $V=4$ respectively

Generally, a valuable value for measuring the association between two variables is the correlation coefficient. The formula for computing the correlation coefficient is given by:

$$r^2 = \frac{[\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})]^2}{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (y_i - \bar{y})^2} \quad (3)$$

Here, $\bar{x}$ and $\bar{y}$ denote the averages of the x-coordinates and y-coordinates of the point set, respectively.

Thus, if given a character contour, after calculating its curvature sum, the scale for smoothing the curvature function can be determined using the formula (3). And then, the points on the character contour, which correspond to the local extrema of the smoothed curvature function, are extracted as the feature point candidates.

3.3 Final Feature Points Detection

Feature point candidates derived from the linear regression contain both true feature points and some false feature points caused by noise. We should preserve the true

feature points and discard false ones. To this end, we develop a new method to determine the final feature points from character contours.

For each feature point candidate p_i , we compute an angle θ according to the region of support formed by its two neighbor feature point candidate p_{i-1} and p_{i+1} . Note that if θ is bigger than 180 degree, we use the value of $360 - \theta$ replace that of θ , this will ensure all the value of θ fall into the region between 0 to 180 degree. Thus, every feature point candidate p_i has a ROS represented by the angle θ . These angles are sorted using the Bubble sort algorithm and represented in the 2D Cartesian system. Taking the serials of the angles as the x-coordinates, and the angles $A(i)$, $i = 0, 1, \dots, n$, as the y-coordinates, the chord C connecting the start point $A(0)$ and the end point $A(n)$ is the principal axis of the data set. The line segment with the max distance $M(A, C)$ from the point $A(i)$ to the principal axis is the sub-principal axis, and the corresponding angle $A(\max)$ is selected as the angle threshold. Those candidate points whose θ are small than $A(\max)$ are preserved as final feature points, since bigger θ means the current point lies on the smoother contour segment, it should not be seen as a feature point. Our experiments show that the method produces satisfactory results for our task at hand. The reason is that the principal axis connects two trivial angles. If choosing $A(0)$ as the angle threshold, all candidate feature points are preserved as the final feature points, and while choosing $A(n)$ means remaining no candidate feature points. The angle $A(\max)$ corresponding to the sub-principal axis is the point with the max distance to the two trivial angles, so it can be selected as the angle threshold.

4 Contour Segment Approximation

With the feature points determined above, a character contour is divided into some contour segments. To convert the characters from bitmap font into outline font, we depict the contour segment with a parametric curve. The parametric curve can be selected as a straight line or a quadric (cubic) Bezier curve according to the bend strength of the contour segment. Our method for approximating the contour segment is similar to that of Sarfraz and Khan's [12], but with some modifications.

To measure the bend strength of a contour segment, the normalized accumulated chord-length parameterization method [13] is adopted. Assuming there are n points in the contour segment, with the first point S , and the last point E , a chord C is formed by connecting S and E . To test the bend strength of the contour segment, an average distance d_{ave} is computed from all points of the contour segment to their corresponding points of the chord. If d_{ave} is smaller than a pre-specified error tolerance T_1 , then a straight line connecting S and E is used to fit the current contour segment; if d_{ave} is smaller than a error tolerance T_1 and bigger than T_2 , a quadric Bezier curve is used to approximate the contour segment; otherwise, a cubic Bezier curve is selected to fit the contour segment.

To represent the contour segments using Bezier curves, the control points for Bezier curve need to be determined. We use the least square fitting to determine the control points of Bezier curves. In the following, we just describe how to obtain the control points of a cubic Bezier curve.

Generally, a cubic Bezier is expressed as follows:

$$Q(t_i) = (1-t_i)^3 p_0 + 3t_i(1-t_i)^2 p_1 + 3t_i^2(1-t_i) p_2 + t_i^3 p_3, 0 \leq i \leq n-1 \quad (4)$$

where P_0, P_1, P_2, P_3 are the four control points of the cubic Bezier, and $P_0=S, P_3=E$. The least square fit is obtained by choosing the P_1 and P_2 , which together with two end points P_0 and P_3 are used as the control points to generate a cubic Bezier to fit the contour segment.

The approximating result is shown in Fig.4-Fig.6, where the blue, and black curves suggest that the contour segments are fitted with quadric Bezier or cubic Bezier.

5 Experimental Results

In this section, part of our test examples are presented to demonstrate the effectiveness of our method. All test calligraphy images were obtained through scanning of published tablet images.

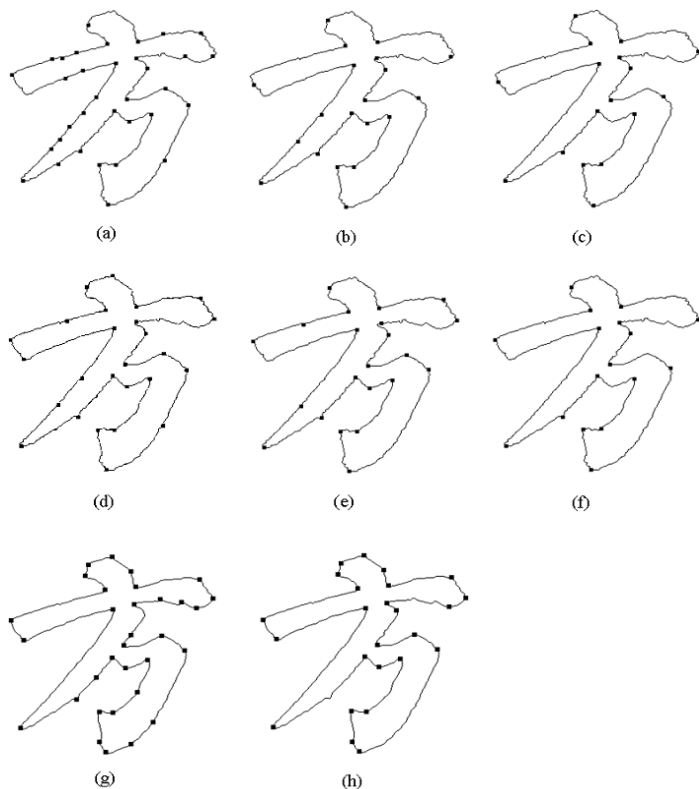


Fig. 3. Chinese character “Fang”: (a)-(c) RJ algorithm with $k=0.02, 0.025, 0.030$ respectively; (d)-(f) RW algorithm with $k=0.025, 0.027, 0.030$ respectively; (g) SD algorithm; (h) our method

The performance of our method for feature-point-detection is compared with that of CSS based method by Sezgin and Davis (SD)[5], and classical corner detection algorithm based on ROS proposed by Rosenfeld and Johnston (RJ)[6], as well as the algorithm by Rosenfeld and Weszka (RW)[7]. In Figs.3(a)-(c), the feature points are detected using RJ algorithm with parameter k varying from 0.02, 0.025 to 0.030 respectively. Some redundant feature points can be found while $k=0.02$. However, the true feature points disappear and false feature points still exist while $k=0.025, 0.030$, respectively. The results shown in Figs.3(d)-(f) with RW algorithm are similar to that of RJ method in terms of true and false feature points detected.

The result of the CSS based algorithm [5] is shown in Fig.3(g). In addition to a few abundant feature points, SD algorithm takes far more time than our method because of many times of convolution involved in it. While our method only need once. The comparison of the elapsed time between the SD algorithm and our method is listed in Table 1.

Table 1. A comparison of elapsed time (in second)

Fig	SD	Our method
Fig.4	104.738	0.138
Fig.5	25.554	0.058
Fig.6	126.305	0.154

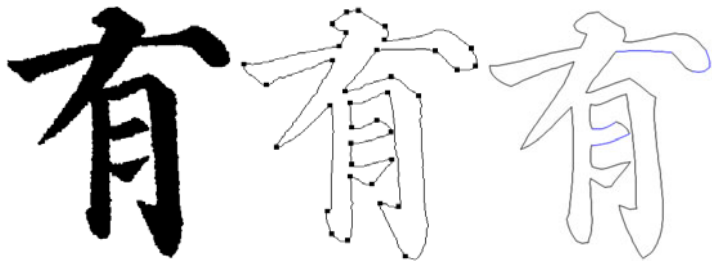


Fig. 4. Chinese character “You”. Left: binary image; Middle: contour and feature points; Right: the fitting result.

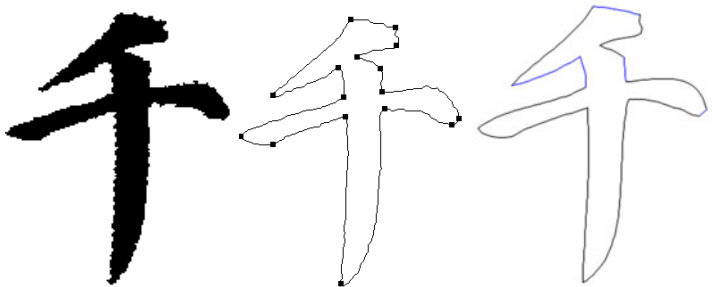


Fig. 5. Chinese character “Qian”. Left: binary image; Middle: contour and feature points; Right: the fitting result.

In Fig.4-Fig.6, the left of each figure is the original binary image, the middle illustrates the detected outlines and feature points, and the right demonstrates the fitting result with parametric curves. Although the three images are noisy on the character's outline, especially the character in Fig.6, our method is able to detect the feature points accurately. And the fitting result of the character outline looks satisfactory.

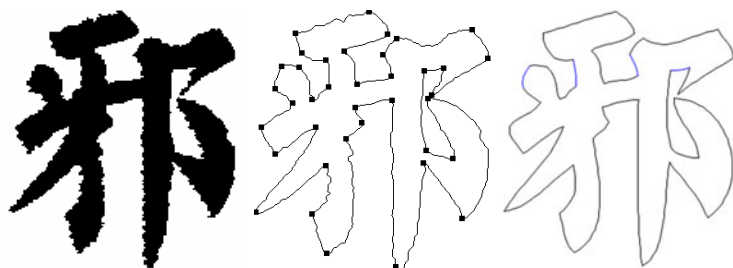


Fig. 6. Chinese character “Xie”. Left: binary image; Middle: contour and feature points; Right: the fitting result.

6 Conclusion

In this paper, a novel method is proposed for outline font generation from historical documents of Chinese calligraphy. The outline font is described with parametric curves approximating outline segments divided by feature points. A statistical deduction method is developed for feature-point-detection from images of calligraphy documents. The proposed method significantly reduces the computational time and is robust to noise compared with previous feature-point-detection methods. The generated outline font preserves the style of the original calligraphic characters well, which can facilitate the further operation of the calligraphic characters, such as using the outline of calligraphic characters in advertising design. We also expect that the system can be used for computer-aided typeface design.

Our currently implemented system works for Kaishu (regular style) well. In addition to Kaishu, there are some other styles such as Xingshu (running style), and Caoshu (cursive style) in Chinese calligraphy. Boundaries of strokes in these styles are rougher than those in Kaishu, because calligraphers wrote them in a fast manner. How to accurately extract the character boundaries and feature points for them is a topic of future work.

Acknowledgments. This paper is an enhanced version of our conference papers [14] [15]. And we thank anonymous reviewers for their very useful comments and suggestions. The work described in this paper was supported by the National Nature Science Foundation of China (No.60903129) and the Natural Science Foundation of Fujian Province of China (No.F0910149).

References

1. Wright, T.: History and Technology of Computer Fonts. *IEEE Annals of the History of Computing* 20(2), 30–34 (1998)
2. Apple Computer Inc., The TrueType Font Format Specification, Version 1.0. (1990)
3. Marji, M.: On the Detection of Dominant Points on Digital Planar Curves, PhD Thesis, Wayne State University, Detroit, Michigan (2003)
4. Rattangsi, A., Chin, R.: Scale-Based Detection of Corners of Planar Curves. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 14(4), 430–449 (1992)
5. Sezgin, T.M., Davis, R.: Scale-space Based Feature Point Detection for Digital Ink. In: *Making Pen-Based Interaction Intelligent and Natural*, pp. 145–151 (2004)
6. Rosenfeld, A., Johnston, E.: Angle Detection on Digital Curves. *IEEE Transactions on Computers* 22(9), 875–878 (1973)
7. Rosenfeld, A., Weszka, J.S.: An Improved Method of Angle Detection on Digital Curves. *IEEE Transaction on Computers* 24, 940–941 (1975)
8. Teh, C.H., Chin, R.T.: On the Detection of Dominant Points on Digital Curves. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 11(8), 859–872 (1989)
9. Witkin, A.P.: Scale-space Filtering. In: *Proc. 8th Int. Joint Conf. Art. Intell.*, pp. 1019–1022 (1983)
10. Canny, J.: A Computational Approach to Edge Detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 8(6), 679–698 (1986)
11. Weisstein, E.W.: Least Squares Fitting, <http://mathworld.wolfram.com/LeastSquaresFitting.html>
12. Sarfraz, M., Khan, M.: An Automatic Algorithm for Approximating Boundary of Bitmap Characters. *Future Generation Computer Systems* 20(8), 1327–1336 (2004)
13. Farin, G.: *Curves and Surfaces for Computer Aided Geometric Design: A Practical Guide*, 4th edn. Academic Press, New York (1997)
14. Zhang, J.S., Lin, H.W., Yu, J.H.: A Novel Method for Vectorizing Historical Documents of Chinese Calligraphy. In: *IEEE International Conference on Computer-Aided Design and Computer Graphics*, pp. 219–224. IEEE Press, Los Alamitos (2007)
15. Zhang, J.S., Yu, J.H., Lin, H.W.: Capturing Character Contours from Images of Ancient Chinese Calligraphy. In: *IEEE 2nd Workshop on Digital Media and its Application in Museum & Heritage*, pp. 36–41. IEEE Press, Los Alamitos (2007)